

ShopAssist

Multi-Repo, Multi-Agent AI Customer-Support Platform

Final Project Report — Architecture Review & Production-Readiness Assessment

Prepared from a full-repository audit spanning all five project repositories:
shopassist · shopassist-client · shopassist-database · shopassist-model · shopassist-devops

July 2026

This report reviews the ShopAssist platform's architecture end to end from an AI engineering / solutions-architect perspective: what was built, how its core AI-engineering concerns (orchestration, RAG, agents, guardrails, observability, evaluation) are implemented, what is strong, what is not yet production-ready, and a prioritized path to close that gap. It also documents a regression-test pass covering five defects fixed during this engagement.

Table of Contents

1. Executive Summary	4
2. Introduction & Scope	4
2.1 Problem Statement	4
2.2 Multi-Repository Platform Overview	5
3. System Architecture	5
3.1 Request Lifecycle	5
3.2 Architectural Flow Diagram	6
3.3 Deployment Topology	6
4. Technology Stack	7
5. Core Concepts Explained	7
5.1 Gateway	8
5.2 Core Orchestration	8
5.3 Retrieval-Augmented Generation (RAG)	8
5.4 Agents	8
5.5 Tools	8
5.6 Observability	9
5.7 Guardrails	9
5.8 Evaluations	9
5.9 CI/CD	9
5.10 Model-Agnostic Design	9
5.11 Plug & Play Modularity	10
5.12 Performance	10
5.13 MCP Readiness	10
6. Strengths	10
7. Weaknesses & Risks	11
8. Stabilization Work Completed This Engagement	12
9. Recommendations — Path to Production	12
9.1 Near Term (Weeks)	12
9.2 Mid Term (1–2 Months)	12
9.3 Long Term (Production Horizon)	13

10. Conclusion

13

1. Executive Summary

ShopAssist is a working, multi-agent, retrieval-augmented customer-support chatbot built as a five-repository platform: a FastAPI orchestration backend (`shopassist`), a Streamlit storefront client (`shopassist-client`), a Postgres/pgvector data layer (`shopassist-database`), a self-hosted model-serving layer for local LLM inference and sentiment/topic classification (`shopassist-model`), and a Docker Compose federation layer that wires all four into one runnable platform (`shopassist-devops`). The system demonstrably works end to end: a customer message is PII-masked, screened for prompt injection, decomposed into sub-tasks, routed to one or more specialist agents that call real tools (an order database, a FAISS-backed RAG store), and synthesized into a natural-language reply that is itself screened before it reaches the customer.

This review covers two things. First, a stabilization pass: five real defects were found and fixed during this engagement — a PII-masking order bug that let a customer's name leak into the shared knowledge base, a garbled/hallucinated support-contact reply from the local fallback model, a customer greeted by name on every single turn instead of naturally, and a Langfuse observability gap in the Docker/FastAPI shutdown path. All five now have regression tests (`tests/test_regressions.py`, 8 tests, all passing) guarding against recurrence — see §8.

Second, a full architectural audit against the AI-engineering concerns that matter for a system putting an LLM in charge of both deciding which tool to call and writing the words a customer reads: gateway, orchestration, RAG, agents, tools, observability, guardrails, evaluation, CI/CD, model-agnosticism, modularity, performance, and MCP readiness (§5). The project's engineering discipline is genuinely strong for its stage — layered guardrails including an IDOR fix and a destructive-action confirmation gate, a real hybrid local/cloud LLM routing design, a working RAGAS-style evaluation harness, and unusually thorough self-documentation across every repository (§6). The gaps that remain are the ones expected of a capstone-stage system moving toward production: no per-repository CI enforcing tests, in-memory state that doesn't survive a restart or scale past one replica, placeholder authentication on both client and server, a provisioned-but-unused pgvector store running in parallel with the FAISS store actually in use, and zero MCP integration (§7). §9 lays out a near/mid/long-term roadmap to close these in priority order.

2. Introduction & Scope

2.1 Problem Statement

ShopAssist answers e-commerce customer-support queries (order status, returns, product recommendations, general policy questions) through a conversational interface, using LLMs both to decide how to handle a request and to write the reply, while keeping a human escalation path for anything it cannot resolve. The engineering challenge this creates — and the lens this report uses throughout — is that an LLM is in the loop for both a state-mutating decision (e.g. cancelling a real order) and the exact words a customer reads, with no separate edge/gateway layer enforcing safety in between. That combination is why guardrails, observability, and evaluation are treated as first-class architectural concerns here, not an afterthought.

2.2 Multi-Repository Platform Overview

Repository	Role	Primary Technology
shopassist	Core orchestration backend: PII masking, guardrails, LLM routing, specialist agents, RAG, evaluation harness	FastAPI, OpenAI SDK (Ollama/Gemini), FAISS
shopassist-client	Customer-facing storefront + chat widget	Streamlit, Web Speech API
shopassist-database	Relational + vector data layer (orders/customers/items, pgvector RAG store)	PostgreSQL 17, pgvector
shopassist-model	Self-hosted model serving: LLM host + sentiment/topic classifier microservice	Ollama, Hugging Face Transformers, FastAPI
shopassist-devops	Cross-repo Docker Compose federation, observability overlay, integration CI	Docker Compose include:, Prometheus, Grafana

Each repository is independently runnable via its own `docker-compose.yml`; `shopassist-devops` layers a shared network on top via Compose's `include:` directive rather than owning application logic itself — a deliberate "plug and play" design discussed further in §5.11.

3. System Architecture

3.1 Request Lifecycle

Every customer message — from either entry point (the CLI simulation or the FastAPI service) — passes through the same pipeline, implemented once in `services/orchestrator.py::AgentOrchestratorService.handle_customer_query()`:

- **PII masking** — regex-based detection of email, phone, self-introduced name, and street-address shapes, replaced before anything reaches an LLM.
- **Sentiment classification** (fail-soft) — calibrates reply tone only; never affects routing.
- **Input guardrail** — screens for blatant prompt-injection/jailbreak phrasing; a hit routes straight to escalation instead of the decomposer.
- **Task decomposition** — an LLM call breaks the message into one or more sub-tasks, each assigned to a named agent; an unrecognized agent name falls back to the general-purpose agent.
- **Agent execution** — each sub-task's agent runs its own reason/tool-call/interpret or RAG-lookup logic (§5.4); any unhandled exception for a sub-task falls back to escalation for that sub-task only.
- **Synthesis** — a final LLM call turns every sub-task's structured result into one coherent reply.
- **Output guardrail + faithfulness check** — the reply is scanned for leaked PII/secrets before the customer sees it, and passively checked for dropped/misstated facts (logged, never blocking).

3.2 Architectural Flow Diagram

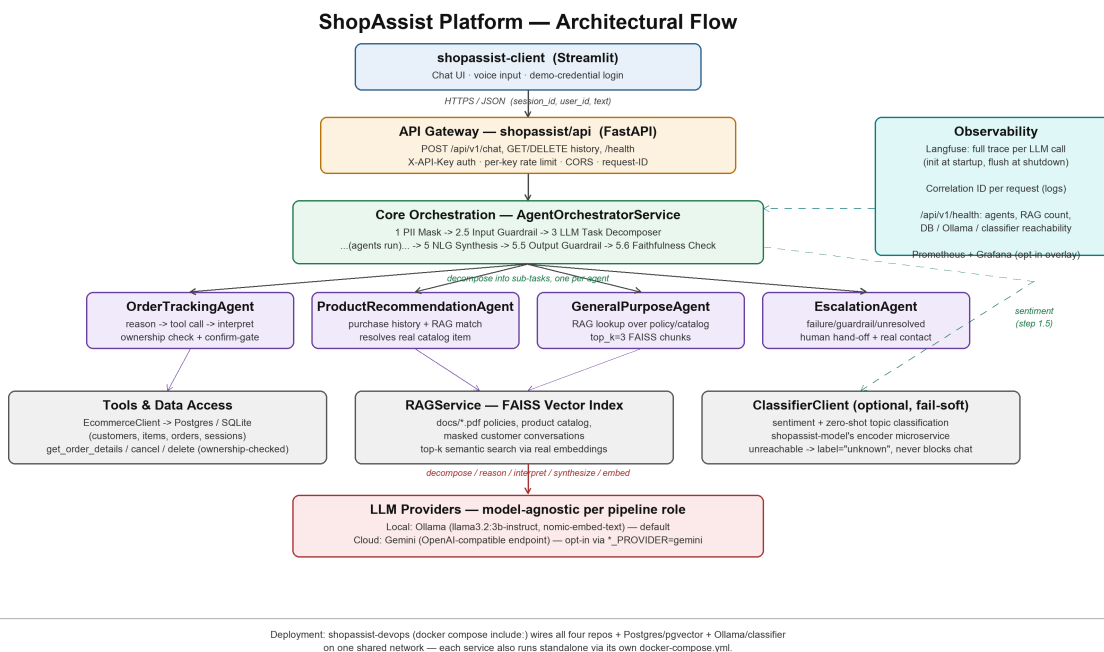


Figure 1. End-to-end request flow: client → gateway → core orchestration → specialist agents → tools/data and RAG → model providers, with observability spanning the gateway and orchestration layers.

3.3 Deployment Topology

In production/demo form, shopassist-devops's root docker-compose.yml brings up all five services in dependency order: Postgres and the model-serving stack (Ollama + classifier) start first with no dependencies, the FastAPI backend waits on both being healthy, and the Streamlit client waits on the backend. Each service also runs standalone against its own compose file for local development — the backend's own compose file, for example, expects an Ollama already running on the host rather than bundling one, keeping that inner loop fast.

4. Technology Stack

Layer	Technology	Notes
API framework	FastAPI 0.139, Uvicorn	Async, OpenAPI/Swagger auto-docs at /docs
LLM access	OpenAI Python SDK 2.47 (protocol, not the vendor)	Same client library against Ollama's and Gemini's OpenAI-compatible endpoints
Local LLM host	Ollama — llama3.2:3b-instruct, nomic-embed-text	CPU-only by default; GPU/vLLM overlay documented
Cloud LLM (opt-in)	Google Gemini (gemini-2.5-flash default)	Per-pipeline-role opt-in via *_PROVIDER=gemini
Classification	Hugging Face Transformers — BERT (sentiment), DeBERTa-v3 zero-shot (topic)	Off-the-shelf checkpoints, not fine-tuned
Vector search	FAISS (IndexFlatL2), in-process	Persisted to a local file, not the provisioned pgvector store
Relational data	PostgreSQL 17 + pgvector extension (SQLite fallback for local dev)	customers / items / orders / sessions / document_chunks
LLM tracing	Langfuse	Full trace per LLM call; init at startup, explicit flush at shutdown
Metrics	prometheus-fastapi-instrumentator, Prometheus, Grafana	Opt-in overlay in shopassist-devops
Frontend	Streamlit 1.46	Server-rendered Python UI; voice input via the browser's Web Speech API
Evaluation	scikit-learn (F1/precision/recall) + custom RAGAS-style metrics	evaluation/ in shopassist
Containerization	Docker Compose (per-repo files + a federating include: layer)	shopassist-devops

5. Core Concepts Explained

This section defines each concept in general terms and states, concretely, how ShopAssist implements it — without repeating the implementation detail already given in §3 or the assessment given in §6/§7, which it cross-references instead of restating.

5.1 Gateway

An API gateway is the single, governed entry point a client talks to — the place authentication, rate limiting, and request shaping happen before a request reaches business logic. ShopAssist does not front its backend with a dedicated gateway product (Kong, Apigee, AWS API Gateway); the FastAPI service itself plays that role — `api/main.py` is literally documented in-repo as "this gateway," exposing `POST /api/v1/chat`, session history endpoints, and `/api/v1/health`, gated by a shared-secret API key, per-key rate limiting, CORS, and a correlation ID stamped onto every request. That collapsing of gateway and application into one process is a reasonable simplification at this scale; see §7 for what it costs at production scale (auth strength, per-process rate limiting).

5.2 Core Orchestration

Orchestration is the component that owns the end-to-end request lifecycle — deciding what happens in what order, and holding the state a single turn needs. `AgentOrchestratorService` is exactly this: one method, described in §3.1, coordinating masking, guardrails, routing, agent execution, and synthesis, and owning the (in-memory) conversation history and pending-confirmation state between turns.

5.3 Retrieval-Augmented Generation (RAG)

RAG grounds an LLM's answer in retrieved, real documents rather than the model's parametric memory alone. ShopAssist's `RAGService` embeds and indexes three sources — policy PDFs, the product catalog, and PII-masked customer conversations — into an in-process FAISS index, queried by real embeddings (Ollama's `nomic-embed-text` locally, or Gemini's embedding model). A second, parallel RAG substrate — a `pgvector` `document_chunks` table with its own ingestion script — is fully provisioned in `shopassist-database` but not read or written by the live application; §7 covers the consolidation this leaves outstanding.

5.4 Agents

An agent, here, is a component that owns one domain of the conversation and decides its own next action rather than following a fixed script. ShopAssist runs four: an order-tracking agent (the fullest reason → tool-call → interpret example, and the only one with write access to real data), a product-recommendation agent, a general-purpose RAG-backed Q&A agent, and an escalation agent that is both the explicit human-handoff path and the implicit safety net every other agent falls back to on failure. A single customer message can fan out to more than one agent in the same turn.

5.5 Tools

Tools are the concrete actions an agent can take against the outside world beyond generating text. Here that means two hardcoded Python integrations: `EcommerceClient` (order/customer/item lookups and mutations against Postgres or SQLite, with ownership checks) and `RAGService`'s query interface. Every tool call is a direct in-process method call decided by an LLM's reasoning step, not a call through a standardized tool-serving protocol — see MCP Readiness below.

5.6 Observability

Observability is the ability to reconstruct what a system did and why, after the fact. ShopAssist layers three kinds: full LLM-call tracing via Langfuse (now correctly initialized and flushed on both the CLI and the Docker/FastAPI shutdown paths — see §8), request-level correlation IDs threaded through every log line, and a dependency-aware `/api/v1/health` endpoint reporting DB/Ollama/classifier reachability independently so one dependency being down doesn't mask the others. An opt-in Prometheus + Grafana overlay in `shopassist-devops` adds generic HTTP-layer metrics (request rate, p95 latency, 5xx rate) across the API and classifier — deliberately scoped to that layer only; it does not yet cover business metrics like RAG hit rate or per-agent routing volume.

5.7 Guardrails

Guardrails are the rule-based checks that don't trust the LLM's judgment alone for the things that matter most. ShopAssist's are notably deliberate for a project this size: input prompt-injection screening, output PII/secret-leak redaction, a faithfulness check on generated replies, and — the two most consequential — an order-ownership (IDOR) check the project's own documentation flags as "the highest-severity gap" once found and fixed, and a confirmation gate in front of any destructive order action so a single LLM decision is never sufficient to cancel or delete a real order. All are plain, synchronous, pattern-based checks by design — no new service or redesign — which is also their limitation; see §7.

5.8 Evaluations

Evaluation is how a system's quality is measured on a fixed basis rather than by spot-checking chat transcripts. ShopAssist's `evaluation/` module runs a hand-labeled case set through the real orchestrator and reports routing accuracy/F1 (with a confusion matrix) and four RAGAS-style RAG metrics — context precision, context recall, faithfulness, and answer relevancy — as a self-contained HTML dashboard. This is real, working infrastructure, not a stub; its main limitation (small dataset, not wired into CI) is a scope-and-integration gap, not a capability gap — see §7 and §9.

5.9 CI/CD

Continuous integration/delivery is what turns "it works on my machine" into a repeatable, enforced guarantee. Today that exists at exactly one layer: a GitHub Actions workflow in `shopassist-devops` that checks out all four application repositories, boots the full platform via Docker Compose, and smoke-tests the chat endpoint end to end — valuable as a "does the platform still boot together" signal, which no single repo's own CI could catch. No individual repository — including this engagement's new regression suite (§8) — currently runs in any per-repo CI pipeline; see §7 and §9.

5.10 Model-Agnostic Design

A model-agnostic system can swap which LLM (or provider) handles a given call without changing calling code. ShopAssist implements this per pipeline role — routing, agent-reasoning, agent-interpretation, generative synthesis, and embeddings each independently default to local Ollama and can be flipped to Gemini via one environment variable each, through a single client abstraction (`LLMInferenceService._resolve_role()`) that also handles the two different structured-output mechanisms each provider actually speaks. A cloud call that fails transparently retries once against the local model rather than failing the turn outright.

5.11 Plug & Play Modularity

Each of the five repositories is independently buildable, runnable, and testable in isolation, and shopassist-devops composes them by reference (`include:`) rather than by copying or forking their configuration — changing one repo's compose file needs no corresponding edit in the federation layer. Optional services follow the same principle at runtime: the sentiment/topic classifier is not required for chat to function at all (it fails soft to `label="unknown"`), and the LLM provider for any given pipeline role is a configuration switch, not a code change.

5.12 Performance

The system's dominant performance characteristic is CPU-only local inference: a single chat turn can make several sequential local LLM calls (decomposition, per-sub-task reasoning/interpretation, final synthesis, embeddings), each taking 30-45 seconds on the reference laptop the project was developed against, occasionally totaling minutes for a multi-intent message. The project's own documentation is candid about this and recommends routing at least the customer-facing synthesis step to Gemini for real usage; the client masks the wait with a generous timeout and a client-side typewriter effect rather than solving the underlying latency. See §7 and §9 for the production path off this default.

5.13 MCP Readiness

The Model Context Protocol (MCP) is an emerging open standard for exposing tools and context to an LLM through a uniform, discoverable interface, decoupling "what tools exist" from "which agent framework is calling them." ShopAssist has no MCP integration today — every tool (`EcommerceClient`, `RAGService`) is a direct, hardcoded Python method call reached only through this codebase's own agent classes. The readiness signal is positive, though: those tools are already cleanly encapsulated behind narrow, well-typed method boundaries (not scattered inline SQL or ad hoc HTTP calls), which is most of the refactor an MCP server wrapper would need — the gap is an integration one, not an architectural rewrite. See §9.

6. Strengths

- **Layered, purpose-built guardrails** around the exact risk this architecture creates — an LLM with real tool-call authority and no separate edge layer — including a fixed IDOR vulnerability and a deterministic (non-LLM) confirmation gate in front of every destructive order action, so a prompt-injection success alone can no longer execute one.
- **Genuine model-agnostic, hybrid-provider design**: every pipeline role independently chooses local vs. cloud, with automatic cloud→local failover on error — not a hardcoded single-provider integration.
- **A real evaluation harness**, not a stub: routing F1/confusion-matrix and RAGAS-style RAG metrics against a hand-labeled case set, producing a shareable HTML dashboard — rare at this project stage.
- **Authentic multi-agent decomposition** with a working reason → tool-call → interpret pattern (`OrderTrackingAgent`) rather than a single monolithic prompt wearing an "agent" label.
- **Real, working observability**: full per-call LLM tracing via Langfuse (correctly lifecycle-managed on both entry points as of this engagement), request correlation IDs, a dependency-aware health endpoint, and an opt-in Prometheus/Grafana overlay.
- **A federated, independently-runnable multi-repo architecture** that is plug-and-play in practice, not just in name: every service is individually buildable, and optional services degrade gracefully rather than hard-failing the platform.

- **Unusually thorough, honest self-documentation** across all five repositories — every README candidly lists its own known gaps and rationale for design trade-offs, which materially accelerated this review and is itself a signal of engineering maturity.
- **A demonstrated bug-fixing and regression-testing discipline:** five concrete defects (including a customer-data-hygiene incident) were root-caused and fixed with an accompanying automated regression suite during this engagement — see §8.

7. Weaknesses & Risks

- **No per-repository CI.** The only automated pipeline is a single cross-repo Docker Compose smoke test in shopassist-devops; no repository — including the backend's new regression suite — runs its own tests, linting, or type-checking on every change. Four of five repositories have no automated tests at all.
- **Placeholder authentication, both ends.** The backend's API-key check is explicitly documented as "not a security boundary you'd trust in production" and is off by default; the client's login accepts any user-ID/password pair where the two happen to match. Together there is no real identity boundary end to end.
- **In-memory, per-process state throughout.** Conversation history, pending-confirmation state, and rate limiting on the backend, and the chat transcript store on the client, all live in plain process memory — none survive a restart, and none work correctly behind more than one replica.
- **Two parallel, unreconciled RAG stores.** A pgvector document_chunks table is fully provisioned and has its own ingestion tooling in shopassist-database, but the live application reads and writes only an independent in-process FAISS index — the durable, shareable store exists and sits unused.
- **Client/backend identity split-brain.** The client's displayed customer profile is a single static mock object, unkeyed by user_id, and disconnected from the backend's real seeded customer records — confirmed concretely (the same demo user_id resolves to different names on each side). Any future personalization work has to resolve this before it can be trusted.
- **Performance is not yet production-shaped.** The default local-inference path makes several sequential CPU-bound LLM calls per turn, tens of seconds each; this is masked client-side (a long timeout, a fake typewriter effect) rather than addressed architecturally for the default configuration.
- **No MCP or comparable tool-serving protocol.** Tools are reachable only as hardcoded method calls from this codebase's own agents, with no standardized, discoverable interface another agent framework or client could reuse.
- **Unpinned model/image references.** Classifier models are referenced by Hugging Face repo name with no revision pin, and the Ollama image tag defaults to latest — a silent upstream change could alter production behavior with no corresponding change in this codebase.
- **Guardrails are pattern-based, not learned.** The input injection screen and RAG-store contamination risk (already demonstrated once — see §8) are only as good as known phrasings; paraphrased or novel attacks are not caught. The project's own documentation flags the already-deployed zero-shot classifier as the natural upgrade path.
- **Evaluation coverage is thin and unenforced.** The hand-labeled case set is small, and nothing in CI gates a change on a minimum evaluation score today.

8. Stabilization Work Completed This Engagement

Five defects were found and fixed prior to this architecture review, each now covered by an automated regression test in `tests/test_regressions.py` (8 tests total, run via `python -m unittest discover -s tests`; all passing at time of writing, with no live LLM, database, or network dependency required).

Symptom	Root Cause	Fix
Chatbot addressed a real customer by a stranger's name ("John Doe")	Demo seed conversations containing that name were ingested into the live RAG store; a lowercase-before-mask ordering bug also silently defeated PII name-masking on that text	Fixed masking order (mask before lowercasing); removed fabricated seed data from the live API's startup ingestion
Reply included garbled/mojibake text in place of a support contact address	The generative prompt asked the model to state that a human would follow up but gave it no real contact value, so the (smaller, local fallback) model fabricated one and degenerated into invalid-token output while doing so	Escalation results now carry the store's real, published support contact for the model to quote verbatim; added an output-side guardrail that detects and replaces degenerate/garbled generation
Every reply opened with "Hi <Name>!" instead of naturally	Full, untrimmed conversation history keeps the customer's name in context every turn, and the system prompt had an anti-repetition rule for the bot's own name but none for the customer's	Added a symmetric instruction: address the customer by name only when it naturally fits, not as a rote prefix on every reply
Langfuse traces could be lost on a Docker container stop	The FastAPI/Docker path initialized Langfuse at startup but relied solely on an <code>atexit</code> hook to flush at shutdown — not guaranteed to run inside Docker's <code>SIGTERM</code> → <code>SIGKILL</code> grace window	The ASGI lifespan's shutdown phase now explicitly flushes Langfuse, mirroring the CLI entry point's own explicit flush-before-exit

9. Recommendations — Path to Production

9.1 Near Term (Weeks)

- Wire per-repository CI (unit tests, linting, type-checking) into GitHub Actions for all five repositories, starting with the new regression suite; keep the existing devops smoke test as the integration layer above it.
- Replace placeholder authentication end to end — real JWT/OAuth2 bearer auth on the backend (already named as the intended upgrade in-repo), with the client's login built against it rather than a name/password-match convention.
- Move conversation history, pending-confirmation state, and rate limiting to a shared store (e.g. Redis) so the backend can run more than one replica.
- Reconcile the client's mock customer profile with the backend's real per-user record, or explicitly and visibly wall it off as demo-only until personalization is in scope.

9.2 Mid Term (1–2 Months)

- Consolidate on one RAG backing store: either adopt the already-provisioned pgvector path end to end and retire the in-process FAISS index, or formally decide FAISS is permanent and remove the unused pgvector plumbing — today's split is pure carried risk with no offsetting benefit.

- Materially expand the evaluation dataset and gate merges/deloys on a minimum routing-F1 and RAG-faithfulness score rather than running evaluation ad hoc.
- Move the default generative path off CPU-only local inference for anything resembling production traffic (Gemini, or the model-serving repo's own documented GPU/VLLM overlay), keeping local Ollama as the offline fallback it already is.
- Upgrade the input guardrail from regex pattern-matching to the zero-shot classifier already running in the platform, as the project's own documentation recommends.

9.3 Long Term (Production Horizon)

- Expose backend tools (order lookup, RAG query) behind MCP so the tool surface is documented, discoverable, and reusable by other agent frameworks or clients without bespoke integration work.
- Pin every model and image reference (Ollama tag, Hugging Face revision) and add quality/drift monitoring beyond today's HTTP-layer-only Prometheus metrics.
- Commission a focused security review given the class of bug already found once (order-ownership IDOR) — the same pattern is worth auditing systematically elsewhere in the tool surface.
- Replace the Streamlit client's DOM-scraping JavaScript patches with a standard web frontend, keeping the backend's HTTP contract stable through the migration — already identified and scoped in the project's own UI architecture assessment.

10. Conclusion

ShopAssist is a substantive, working demonstration of a guardrailed, multi-agent, RAG-grounded customer-support system, built with a level of architectural intentionality — particularly around safety (IDOR fix, confirmation gates), provider-agnosticism, and evaluation — that exceeds what its capstone framing might suggest. The defects fixed during this engagement were real but narrow, and are now regression-tested. What stands between this system and a production deployment is not a redesign: it is the standard, well-understood list in §9 — enforced testing, real auth, shared state, one RAG store instead of two, and a faster default inference path — each independently addressable without disturbing the architecture that already works.